1. Sistemas distribuidos

Grado en Ingeniería Informática y Tecnologías Virtuales
Grado en Ingeniería del Software

Software de Comunicaciones

Curso 2025-2026

Universidad LOYOLA

Contenido

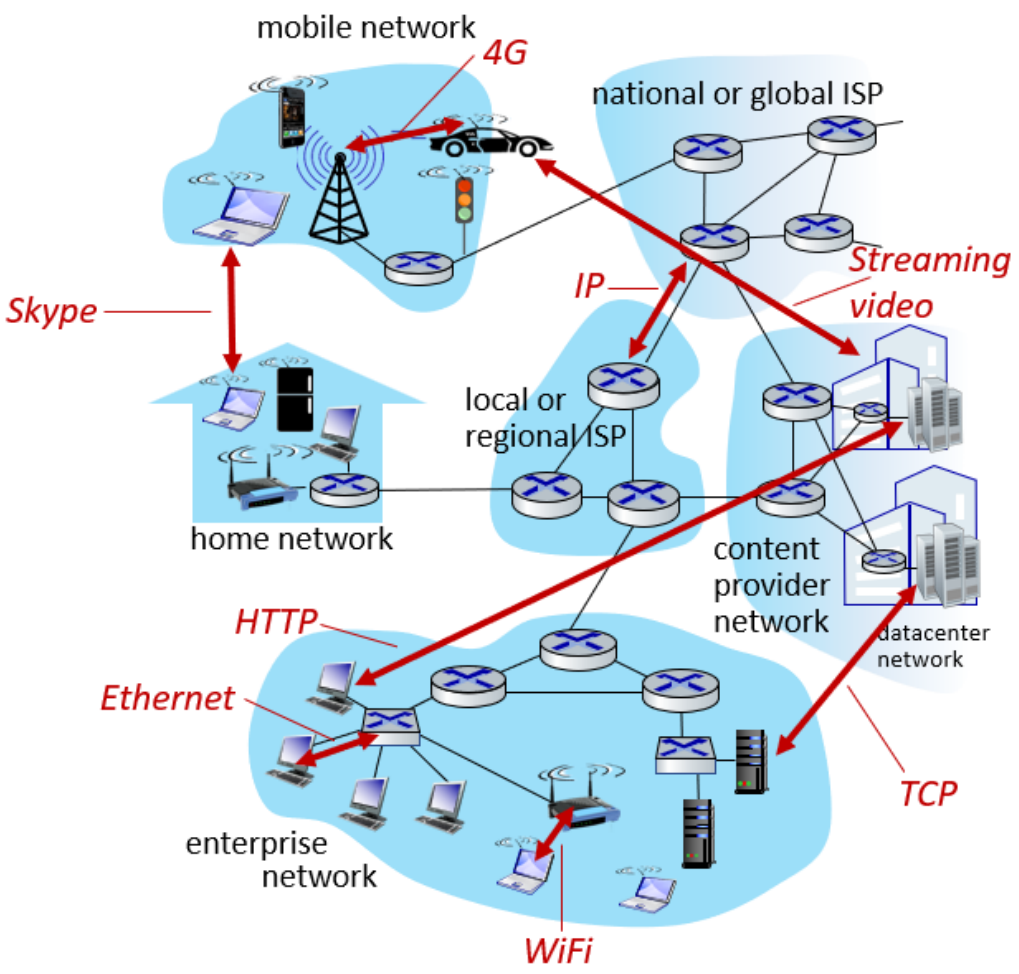
1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

Contenido

1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

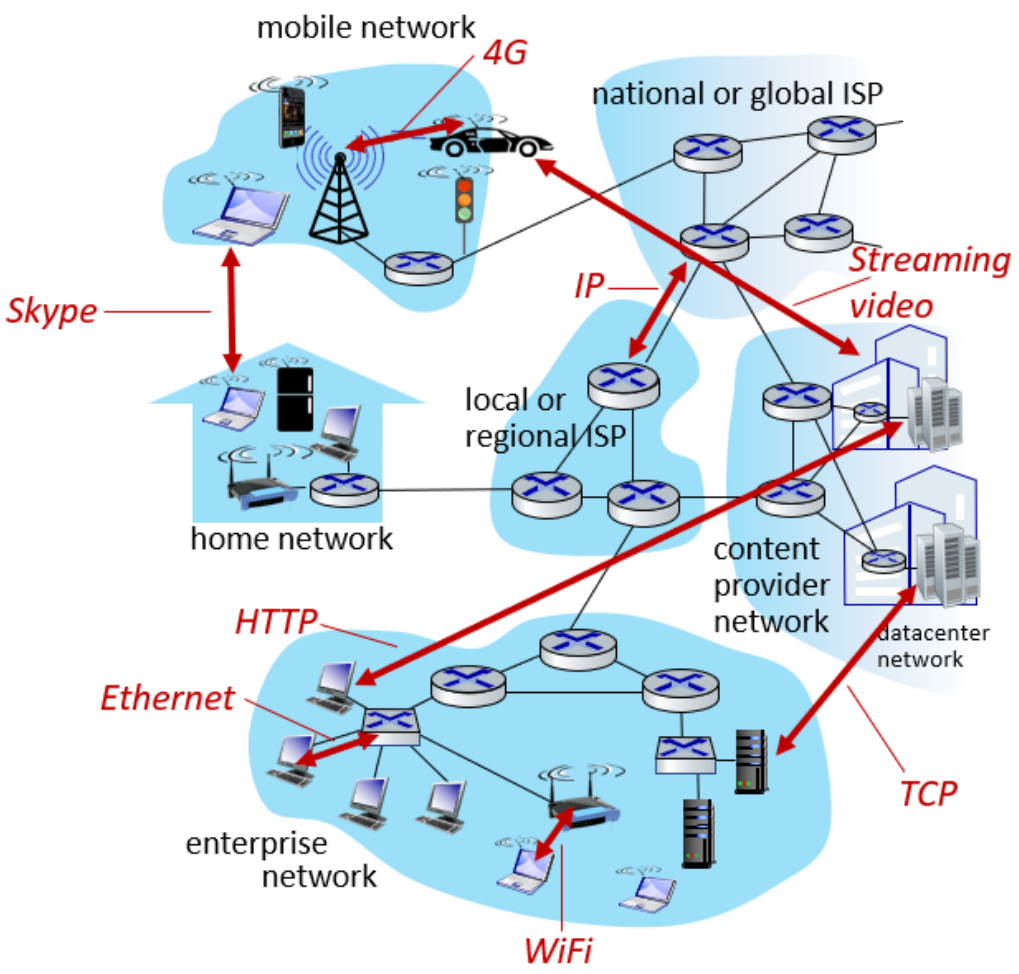
Caracterización de los sistemas distribuidos

- Un **sistema distribuido** es un conjunto de dispositivos independientes que ofrece a sus usuarios la imagen de un sistema único y coherente.
- Las aplicaciones se comunican mediante **paso de mensajes** para poder sincronizarse y cooperar.
- **Internet** es el sistema distribuido de ingeniería más grande creado por la humanidad. Es una red de computadoras que interconecta cientos de millones de dispositivos a través del mundo. Es una infraestructura que proporciona servicios a aplicaciones (programas en las computadoras)
- Ejemplos: servicios de búsqueda, sistemas multimedia, juegos online, comercio electrónico, sistema de archivos en red, redes de telefonía móvil, redes corporativas, clúster de computadoras para supercomputación, internet en general…



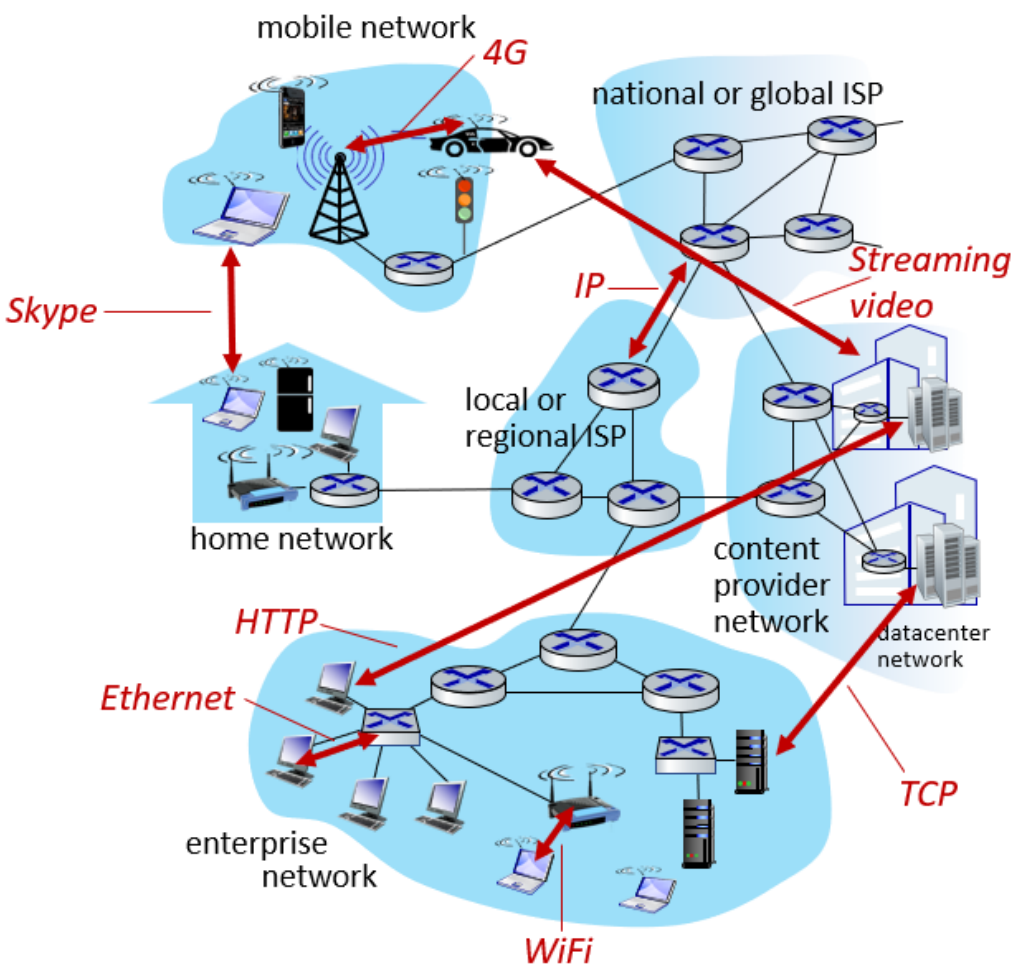
Caracterización de los sistemas distribuidos

- Los **sistemas finales** o ***hosts*** ejecutan aplicaciones que pueden acceder a recursos de otros dispositivos (intercambio de datos).
- Los sistemas finales están conectados entre sí mediante **enlaces de comunicaciones** y **conmutadores de paquetes** (routers y *switches*).
- Los sistemas finales acceden a internet a través de **ISPs** (*Internet Service Providers*).



Caracterización de los sistemas distribuidos

- Cada elemento de internet (*hosts*, routers…) ejecutan **protocolos** que controlan el envío y recepción de información dentro de un sistema distribuido.
- Dada la importancia de los protocolos, es importante que todo el mundo esté de acuerdo en lo que cada protocolo hace de forma que todo el mundo pueda crear sistemas y productos que puedan interactuar. Aquí aparecen los estándares. Los estándares de internet son desarrollados por el **IETF** (*Internet Engineering Task Force*). www.ietf.org
- Los documentos del IETF se llaman **RFCs** (*Requests for Comments*).



Caracterización de los sistemas distribuidos

Los **sistemas distribuidos** se caracterizan por lo siguiente:

- **Concurrencia**: la ejecución de programas concurrentes es la norma. Cada uno puedo hacer trabajos independientes en su ordenador mientras se comparten recursos como base de datos, archivos o páginas web.
- **No hay un reloj global:** cuando los programas necesitan cooperar, coordinan sus acciones por medio de **mensajes** (afectados por retrasos, pueden sufrir fallos y son objeto a ataques de seguridad). La coordinación cercana a veces depende de una idea del momento en el cual la acción del programa va a ocurrir, pero existen límites en cuanto a la precisión en la que los computadores pueden sincronizar sus relojes.
- **Independencia a fallos:** Cualquier sistema computacional puede fallar. Si un elemento de un sistema distribuido falla, el sistema completo debe seguir en funcionamiento.

Contenido

1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

Retos de los sistemas distribuidos

1. Heterogeneidad
2. Apertura
3. Seguridad
4. Escalabilidad
5. Manejo de fallos
6. Concurrencia
7. Transparencia
8. Calidad del servicio

Retos de los sistemas distribuidos

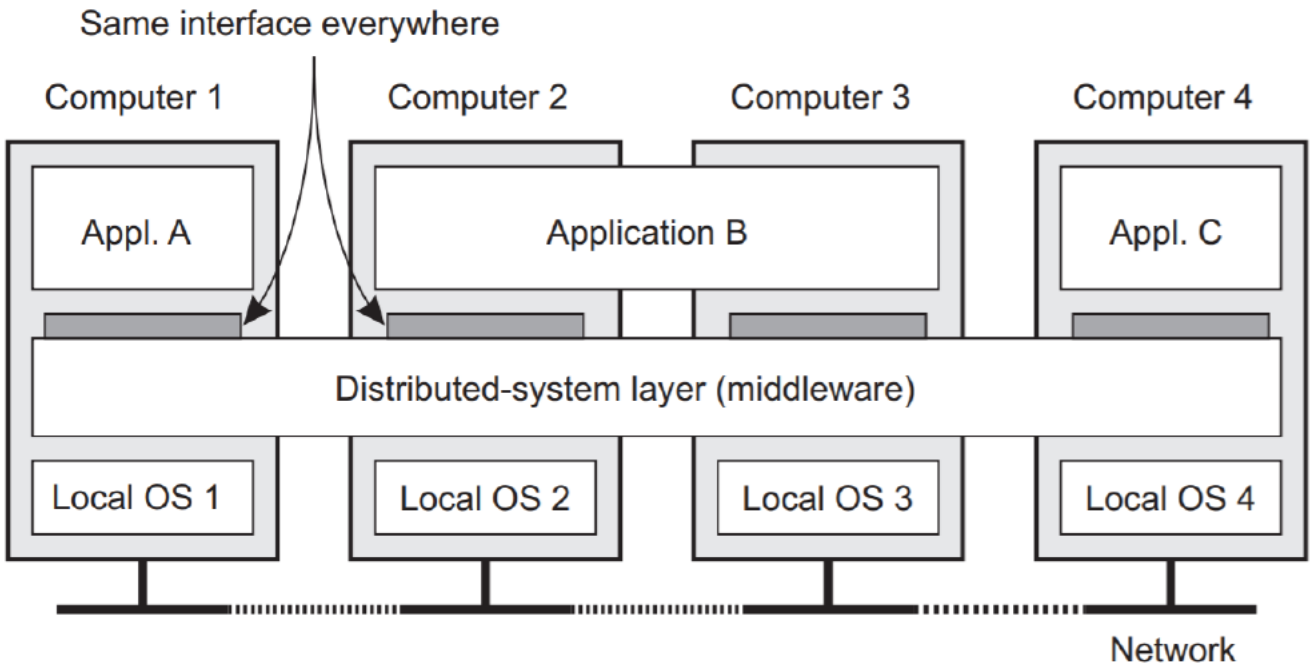
1. Heterogeneidad

Acceso a servicios y ejecución de aplicaciones sobre una colección heterogénea de computadores y redes (diferencias en hardware, enlaces de comunicación, sistemas operativos, lenguajes de programación, implementación por diferentes desarrolladores…).

Middleware

Capa de software que proporciona abstracción y enmascara la heterogeneidad de los dispositivos subyacentes.

Proporciona un modelo computacional uniforme para programadores de servidores y desarrolladores de aplicaciones distribuidas.



Retos de los sistemas distribuidos

2. Apertura

- Característica que determina si el **sistema** puede ser **extendido** y **reimplementado** en diversas formas.
- Se consigue con la estandarización de interfaces, aunque es el punto inicial. El reto para los diseñadores es afrontar la complejidad de los sistemas distribuidos que consisten en multitud de componentes diseñados por diferentes agentes.
- Un sistema abierto es aquel que se ajusta a estándares que especifican y describen la sintaxis y la semántica de las operaciones que constituyen el servicio ofrecido por el sistema.
- Las RFCs no son el único medio para la estandarización. Por ejemplo, el ***World Wide Web Consortium*** **(W3C)** desarrolla y publica estándares relacionados con la web. www.w3.org.

Retos de los sistemas distribuidos

3. Seguridad

La información en sistemas distribuidos tiene una gran importancia para los usuarios. 3 componentes:

- Confidencialidad
- Integridad
- Disponibilidad

Retos de los sistemas distribuidos

4. Escalabilidad

Los sistemas distribuidos operan de forma eficiente y efectiva en diferentes escalas.

Un sistema es escalable si permanece efectivo cuando hay un incremento significante en el número de recursos y número de usuarios.

El diseño de sistemas escalables presenta los siguientes retos:

- Control del **coste** de los recursos físicos.
- Control de la perdida de **rendimiento.**
- Prevenir que los recursos de **software** se agoten.
- Evitar **cuellos de botella**. En general los algoritmos deben ser descentralizados.

Retos de los sistemas distribuidos

5. Manejo de fallos

Las siguientes técnicas pueden ser usadas:

- **Detección de fallos:** por ejemplo, uso de *Checksums*, códigos de Hamming… para detectar mensajes corruptos.
- **Enmascarar fallos:** mensajes pueden ser retransmitidos cuando fallan al llegar. Archivos pueden ser escritos en diferentes discos, de forma que si uno es corrupto el otro puede aún ser correcto.
- **Tolerar fallos:** por ejemplo, informar de que un cliente no se ha conectado a un servidor, pudiendo intentar la conexión de nuevo.
- **Recuperación después de fallos:** por ejemplo, si un servidor falla y no ha finalizado una tarea.
- **Redundancia:** replicación de componentes.

Retos de los sistemas distribuidos

6. Concurrencia

Uno de los problemas a resolver cuando se implantan programas concurrentes es garantizar que las secciones críticas existentes (recursos compartidos, por ejemplo, una base de datos) se ejecuten en exclusión mutua, es decir que únicamente las ejecute un proceso en cada momento.

Se debe asegurar su correcto comportamiento.

Técnicas: semáforos

Retos de los sistemas distribuidos

7. Transparencia

Un **sistema distribuido** es **transparente** si es capaz de ocultar que físicamente está compuesto por múltiples elementos independientes. El sistema es percibido **como un todo** en lugar de una colección de elementos independientes.

Transparencia en	Descripción
Acceso	Permite a **recursos** locales y remotos ser **accesibles** usando operaciones idénticas
Fallos	**Oculta** el **fallo** y la recuperación de los recursos permitiendo a las aplicaciones realizar sus tareas
Migración	**Oculta** el hecho de que un **recurso** pueda migrar de un lado a otro sin afectar la operación de usuarios y programas
Concurrencia	Varios **procesos** operando usando **recursos compartidos** sin interferencia entre ellos
Replicación	**Múltiples recursos** sin necesitad que los usuarios o programas lo conozcan
Rendimiento	Permite al **sistema** ser **reconfigurado** para mejorar rendimiento en función de la carga
Escalado	Permite al sistema y las aplicaciones **expandirse** en **escala** sin cambiar la estructura del sistema ni los algoritmos de aplicación
Ubicación	Permite a los recursos ser **accesibles sin conocimiento** de su **localización** física o red

Retos de los sistemas distribuidos

8. Calidad del servicio

Propiedades no funcionales que afectan el servicio:

Fiabilidad

Rendimiento

Seguridad

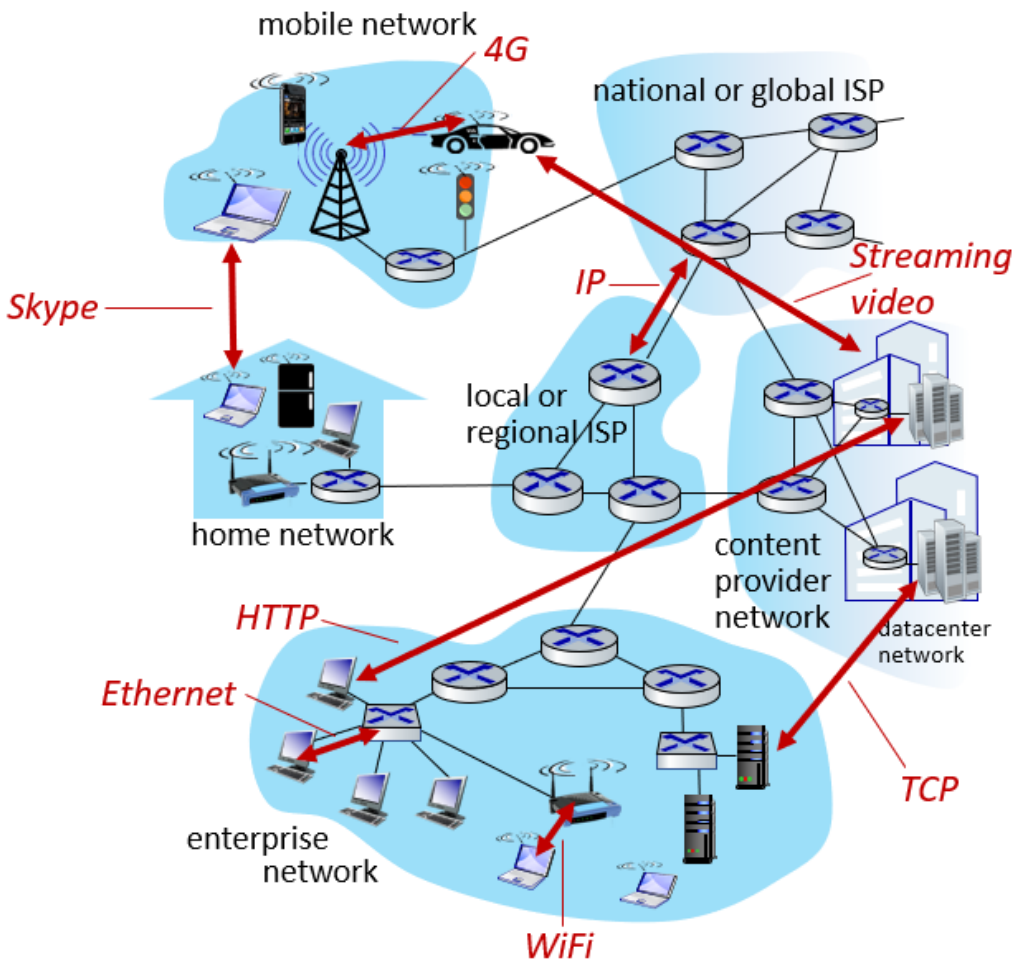
Contenido

1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

Arquitectura

Los **sistemas distribuidos** son **complejos** con diversos elementos. En redes de computadores encontramos:

- *Hosts*
- *Hubs*
- *Switches*
- *Routers*
- Enlaces físicos de diferentes naturalezas
- Aplicaciones
- Protocolos
- Hardware, software



Arquitectura

¿Hay alguna forma de organizar la estructura de una red?

RFC 3439: *Some Internet Architectural Guidelines and Philosophy*

https://www.rfc-editor.org/rfc/rfc3439

RFC 1958: *Arquitectural Principles of the Internet*

https://www.rfc-editor.org/rfc/rfc1958

Arquitectura

¿Hay alguna forma de organizar la estructura de una red?

RFC 3439: *Some Internet Architectural Guidelines and Philosophy*

https://www.rfc-editor.org/rfc/rfc3439

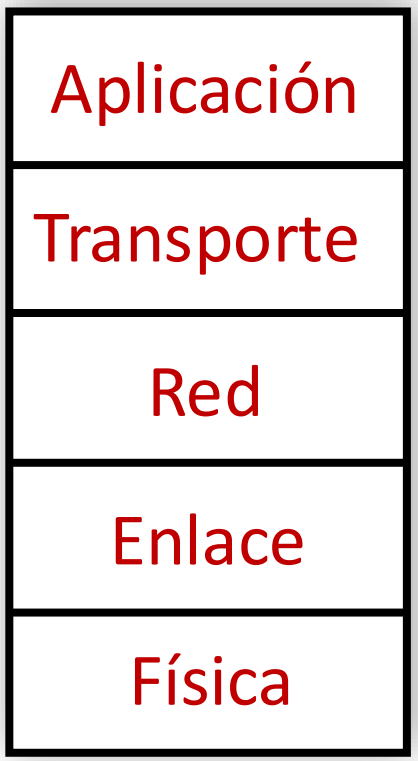
RFC 1958: *Arquitectural Principles of the Internet*

https://www.rfc-editor.org/rfc/rfc1958

Modelo en capas ***presenta varias ventajas conceptuales y estructurales***

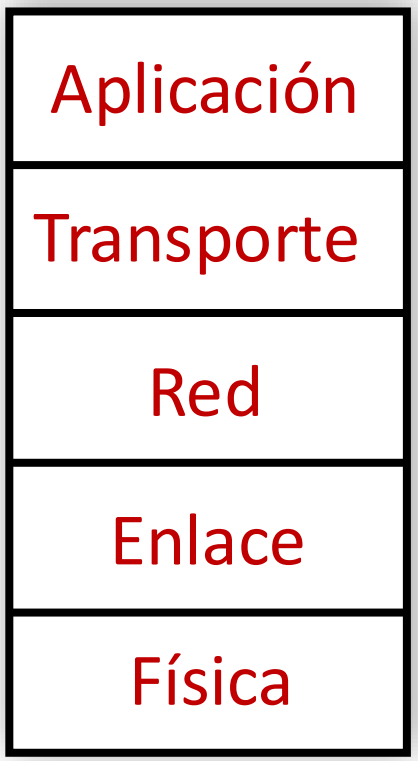
Arquitectura: modelo de capas

- *Aplicación:* aplicaciones de red
  - HTTP, IMAP, SMTP, DNS, FTP, NTP, DHCP
- *Transporte:* transferencia de datos entre procesos
  - TCP (entrega ordenada de mensajes), UDP
- *Red:* enrutado de datagramas desde la fuente al destino
  - IP, ARP, ICMP, IGMP
- *Enlace:* transferencia de datos entre vecinos
  - Ethernet, 802.11 (WiFi), PPP
- *Física:* bits en el medio físico



Arquitectura: modelo de capas

- Para gestionar las comunicaciones es necesario definir una **pila de protocolos.** Cada nivel utiliza un protocolo determinado para solucionar algún problema que conlleva la comunicación.
- **¿Por qué un modelo en capas?**
  - Enfoque para diseñar y discutir sobre **sistemas complejos.**
  - **La modularización** facilita el mantenimiento y la actualización del **sistema**. Cambios en la implementación de un servicio de capa es transparente al resto del sistema.



Arquitectura: modelo de capas

Aplicación
Transporte
Red
Enlace
Física

M

Aplicación intercambia mensajes para implementar servicios de aplicación usando servicios de la capa de transporte

H_t | M

El protocolo de la capa de transporte transfiere M desde un proceso a otro usando servicios de la capa de red

El protocolo de la capa de transporte encapsula el mensaje de la capa de aplicación, M, con la cabecera H_t para crear un segmento de la capa de transporte. H_t es usado por la capa de transporte para implementar su servicio

Aplicación
Transporte
Red
Enlace
Física

Arquitectura: modelo de capas

Aplicación	
Transporte	
Red	
Enlace	
Física	

$\longleftarrow$ M $\longrightarrow$

$\longleftarrow$ H_t | M $\longrightarrow$

El protocolo de la capa de transporte transfiere M desde un proceso a otro usando servicios de la capa de red

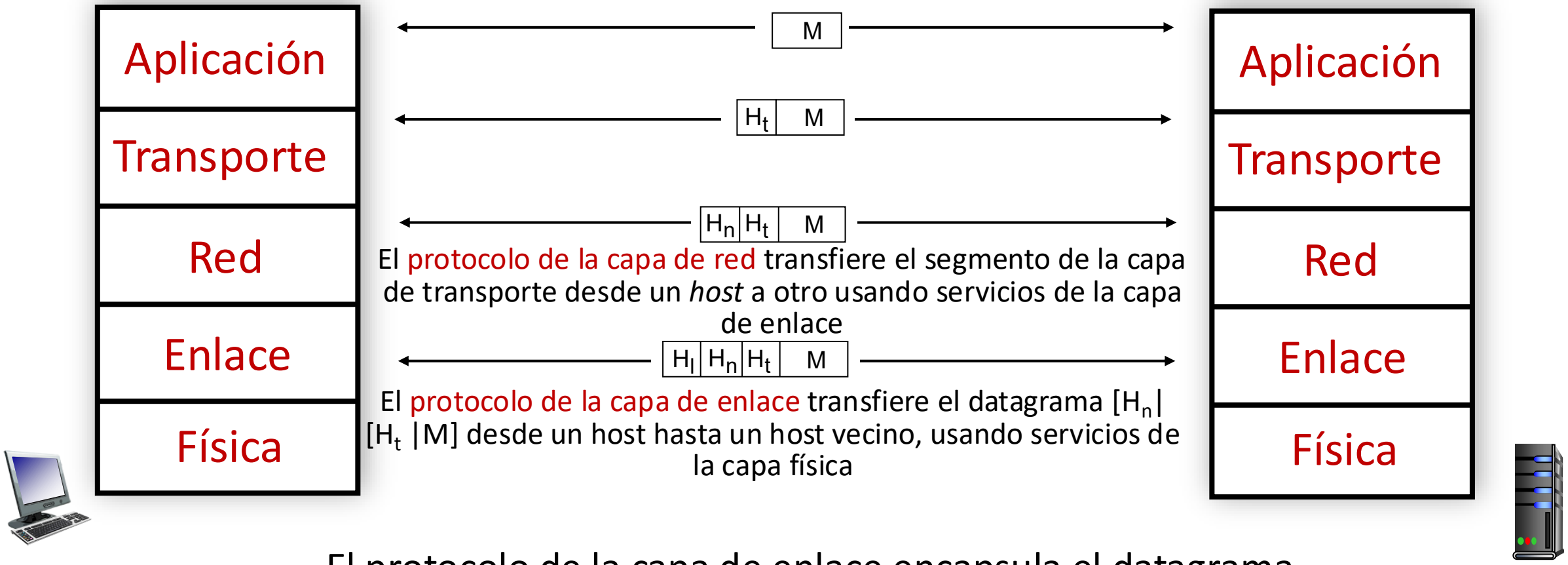
$\longleftarrow$ H_n | H_t | M $\longrightarrow$

El protocolo de la capa de red transfiere el segmento de la capa de transporte desde un *host* a otro usando servicios de la capa de enlace

El protocolo de la capa de red encapsula el segmento de la capa de transporte [H_t | M] con la cabecera H_n de la capa de red para crear un datagrama de la capa de red. H_n es usado por el protocolo de la capa de red para implementar sus servicios

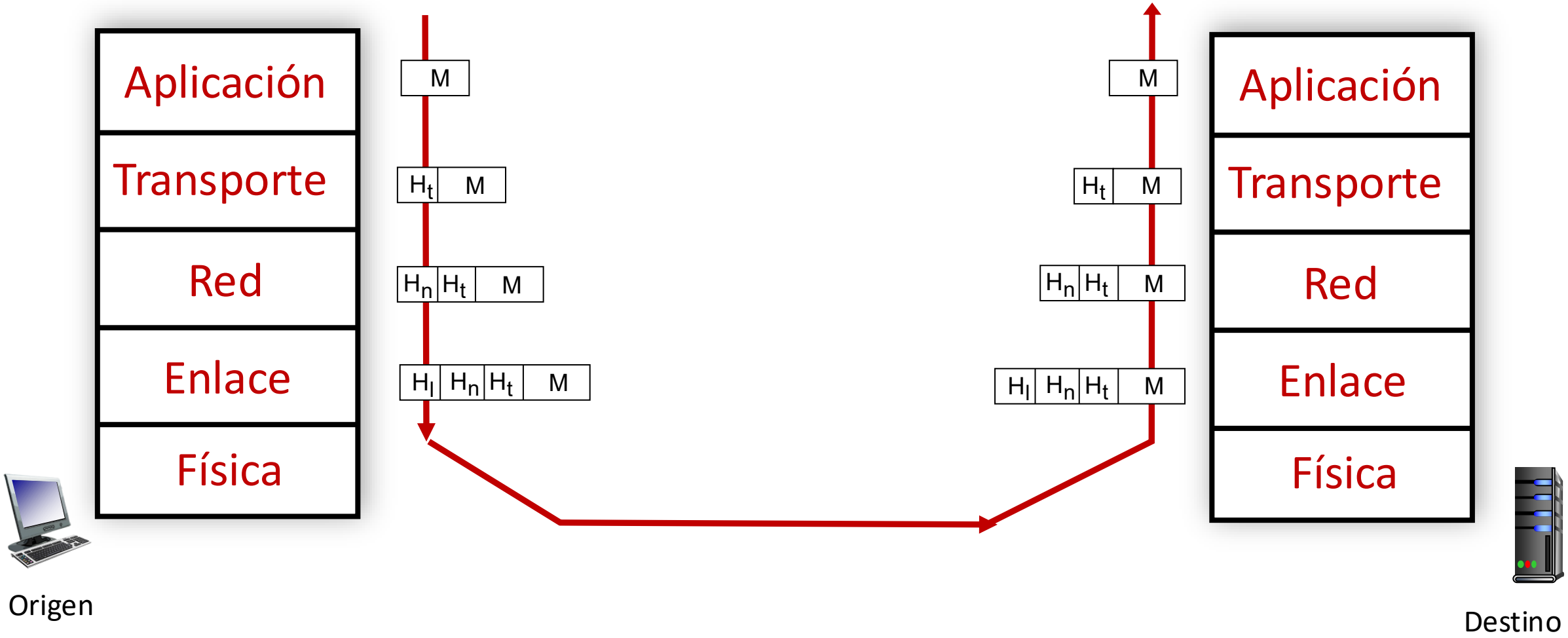
Aplicación	
Transporte	
Red	
Enlace	
Física	

Arquitectura: modelo de capas

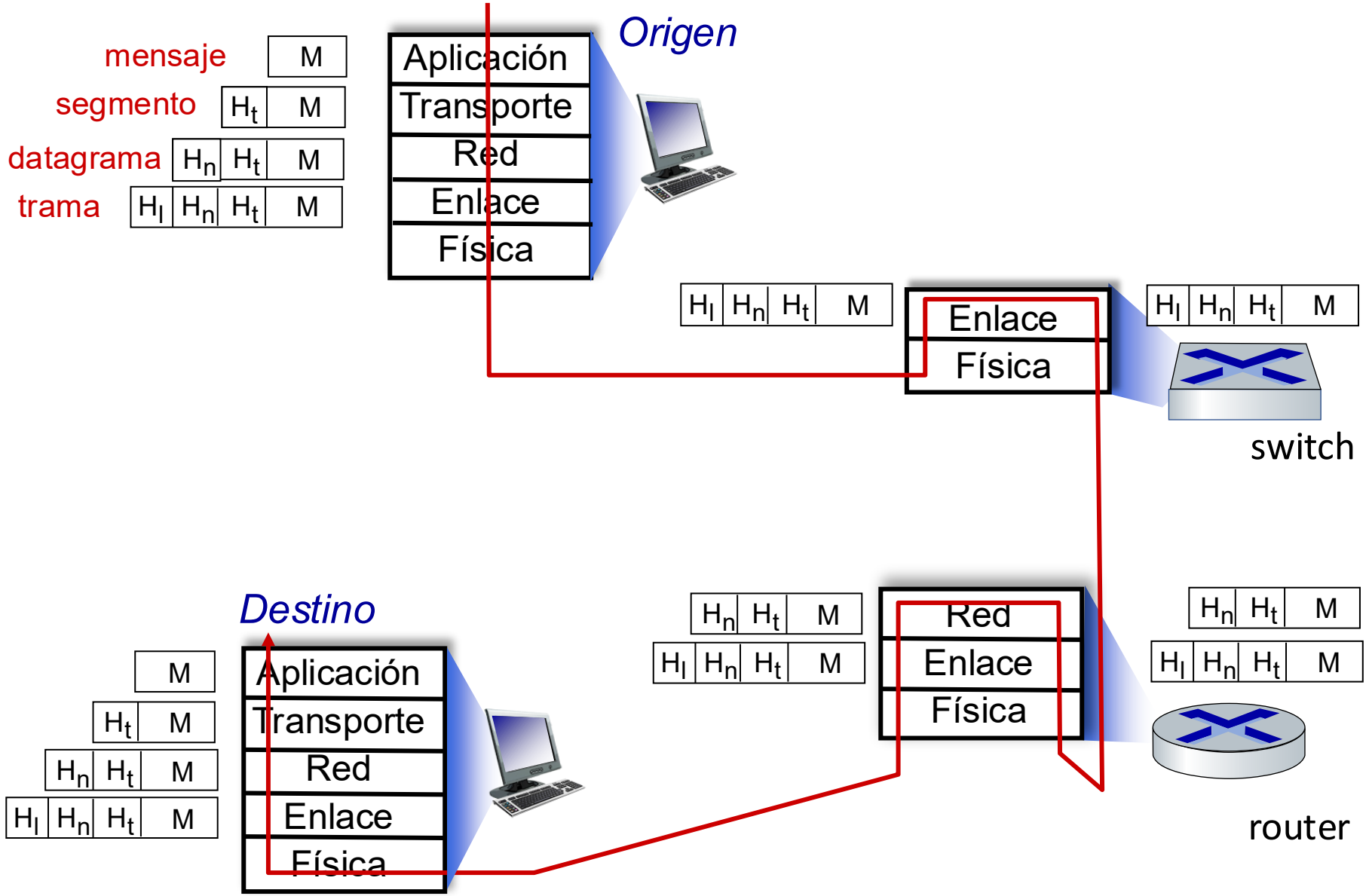


El protocolo de la capa de enlace encapsula el datagrama de red [H_n| [H_t |M] con la cabecera de la capa de enlace H_l para crear una trama de la capa de enlace

Arquitectura: modelo de capas



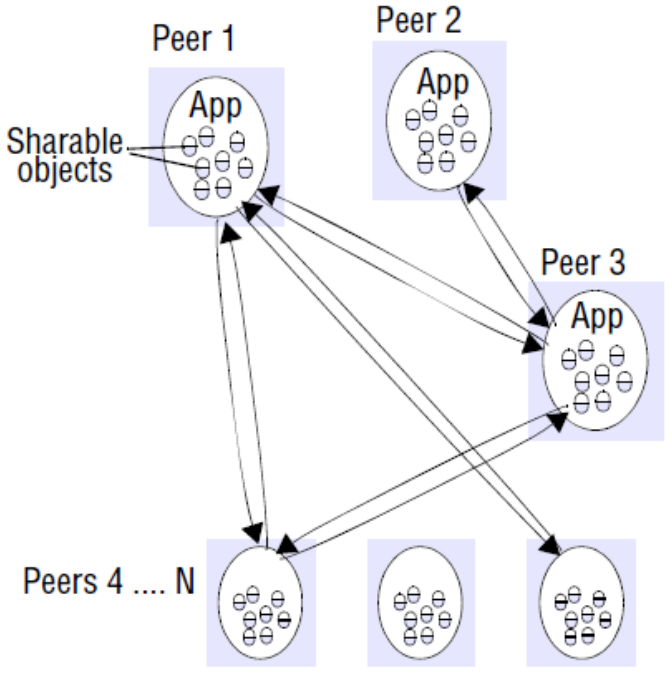
Arquitectura: modelo de capas

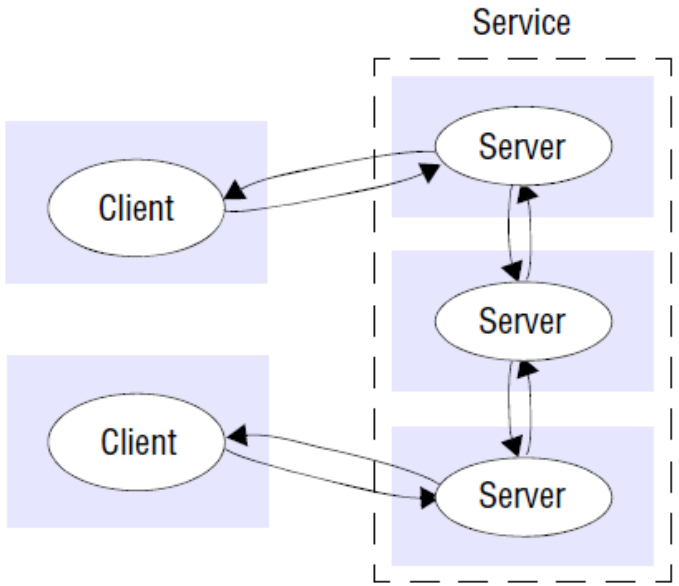


Arquitectura

¿Qué papel y responsabilidad tienen los procesos (programas en ejecución) en la arquitectura global?

- **Cliente-servidor:** los procesos toman el papel de ser clientes o servidores.
- ***Peer-to-peer*:** todos los procesos involucrados en una tarea tienen papeles similares, interaccionan de forma cooperativa sin ninguna distinción entre procesos cliente y servidor. En términos prácticos, todos los procesos participantes ejecutan el mismo programa y ofrece el mismo juego de interfaces para cada uno.





Contenido

1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

Coordinación

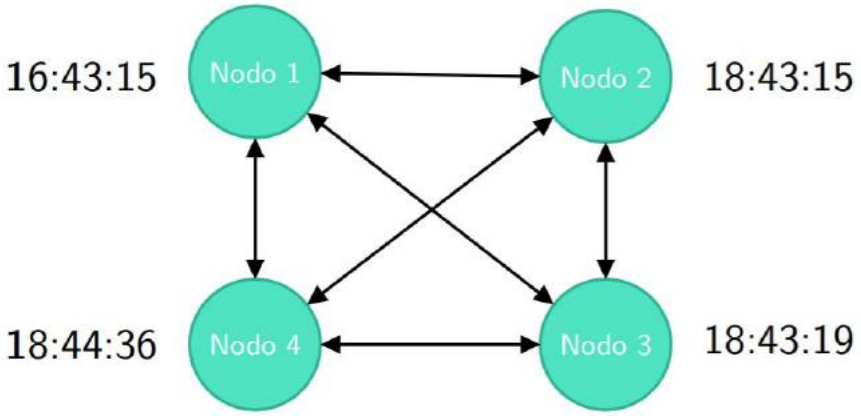
1. Sincronización de reloj
   - Algoritmo de Cristian
   - NTP
2. Exclusión mutua
   - Algoritmo centralizado
   - Algoritmo distribuido
   - Algoritmo *token ring*

Coordinación

1. Sincronización de reloj
   - Algoritmo de Cristian
   - NTP
2. Exclusión mutua
   - Algoritmo centralizado
   - Algoritmo distribuido
   - Algoritmo *token ring*

Sincronización de reloj

- De manera general, el término **sincronización** hace referencia al **conjunto de mecanismos** que se utilizan para **coordinar múltiples actividades** en un determinado sistema o aplicación.
- En el caso de un sistema distribuido, la sincronización plantea algunos problemas adicionales, pues no podemos confiar en la existencia de **ningún reloj** que todos los procesos puedan consultar.
- Debido a esto, ni siquiera resulta sencillo determinar en qué orden llegaron a suceder algunos eventos, si estos fueron ejecutados por procesos ubicados en distintos ordenadores.



Sincronización de reloj

- Buscamos utilizar un algoritmo que consiga **sincronizar** entre sí los **relojes locales** utilizados en cada uno de los nodos.
- El **objetivo** es **aproximar** lo **máximo posible** el reloj de cada máquina **a la hora real**.
- Con esto se garantiza que:
  - Todos los relojes progresan al mismo tiempo.
  - Todo proceso pueda utilizar su reloj local para etiquetar la ocurrencia del evento.
- **Problema**: aunque el tiempo de transmisión en las redes actuales cada vez disminuye más (redes más rápidas), la **frecuencia de los procesadores actuales**, así como el número de instrucciones que pueden ejecutar en cada unidad de tiempo, **también crece muy deprisa**.
- Aunque los **relojes actuales** son relativamente precisos, todavía distan mucho de ser perfectos.
- Los relojes presentan **ligeras desviaciones** en su progreso respecto a la hora real y eso puede generar importantes diferencias cuando se ejecutan operaciones en ordenadores con microprocesadores diferentes.

Sincronización de reloj

- Normalmente un **ordenador** tiene un **temporizador** que genera interrupciones con cierta frecuencia y que el sistema operativo programará adecuadamente para llevar una gestión adecuada del instante actual.
- Así, cuando el ordenador arranque, el **sistema operativo averiguará** cuál es la **hora actual** e inicializará de acuerdo con ésta el valor del contador.
- Por ejemplo, en la mayoría de los sistemas UNIX, el contador recoge el número de *ticks* desde el 1 de Enero de 1970 a las 00:00:00 horas y avanza a unos 60 o 100 *ticks* por segundo.
- Si todos los temporizadores utilizados fueran perfectos, en todo instante, todos los ordenadores de un sistema distribuido tendrían relojes con el mismo valor para su contador local. Sin embargo, los chips de reloj actuales tienen un error relativo.
- Por ejemplo, si el temporizador está programado para generar 60 interrupciones por segundo, debería generar 216.000 *ticks* por hora. Un temporizador convencional, con un error relativo del 0,001 % generará entre 215998 y 216002 interrupciones.
- Los **algoritmos de sincronización** de relojes **tratan** de **ajustar** estos **desfases**.

Sincronización de reloj

- Los **algoritmos de sincronización** están basados en **el intercambio de mensajes** entre los nodos que quieran sincronizar sus relojes. La transmisión de un mensaje por la red consumirá tiempo y será imposible medir con exactitud cuánto tiempo se ha invertido en ese paso, pues los relojes utilizados en el emisor y en el receptor todavía no están sincronizados.
- Los relojes utilizados en cada máquina no deberían retroceder nunca. En caso contrario, algunas aplicaciones podrían funcionar mal. Un ejemplo sería la utilidad *make* que examina los instantes de última modificación de una colección de ficheros.

Algoritmo de Cristian

Este algoritmo asume que existe un ordenador cuyo reloj es preciso (será el **servidor**). Los que quieren sincronizarse con él serán sus **clientes**:

1- El cliente pide el valor del reloj al servidor en el instante T_0, según el reloj local del cliente C_c.

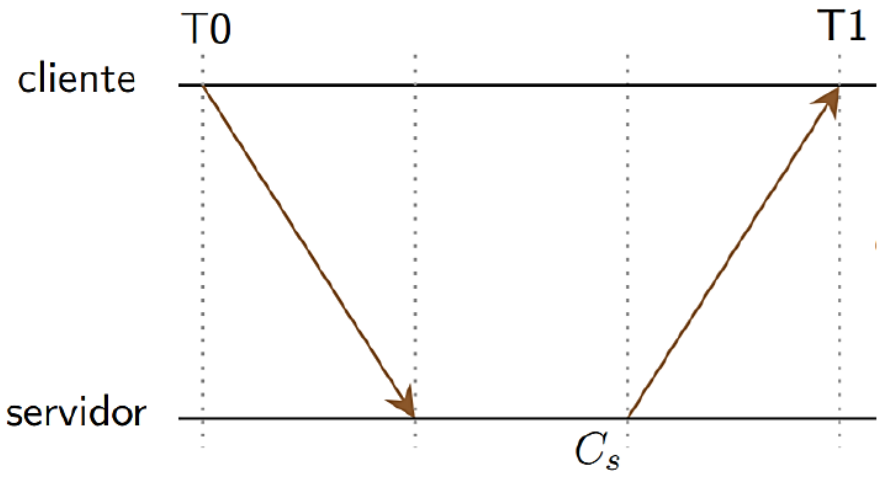
2- El servidor recibe la petición y responde con el valor que tenga su reloj, C_s justo antes de enviar el mensaje de respuesta.

3- La respuesta llega al cliente en T_1, según el reloj local del cliente.

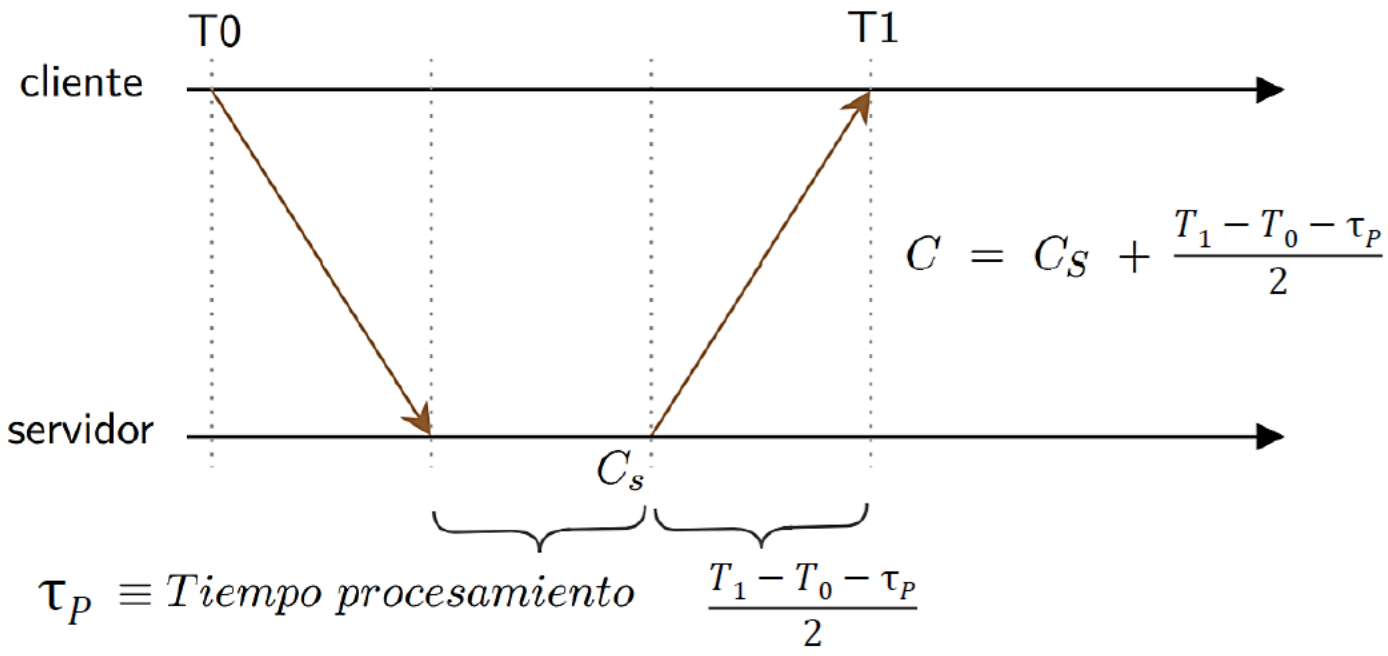
4- El cliente calcula el valor al que debería poner su reloj (C) según:

$$C = C_s + \frac{T_1 - T_0 - \tau_p}{2}$$

5- Si $C>C_c$, entonces el cliente establece su reloj local al valor C. En caso contrario, se descartarán algunos ticks de reloj según C_c-C.



Algoritmo de Cristian



Nota: Este algoritmo asume que el tiempo invertido en la transmisión del mensaje de petición será similar al invertido en la transmisión del mensaje de respuesta. Si la diferencia entre esas dos transmisiones es importante, la sincronización efectuada será imprecisa.

NTP

NTP (*Network Time Protocol*) define una arquitectura para información de tiempo distribuida en Internet. Actualizado en **RFC 7822.**

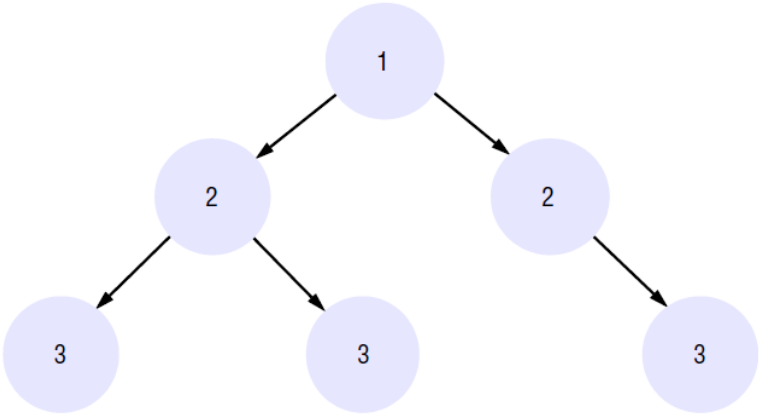
El servicio **NTP** es proporcionado por una **red de servidores** localizados en diferentes puntos de Internet. Los servidores primarios (nivel 1) están conectados con una fuente de tiempo y los secundarios están sincronizados con los primarios.

Los servidores conectados en una **subred de sincronización** en diferentes niveles o estratos (las flechas indican control de sincronización).

NTP tiene en cuenta los retrasos de los mensajes.

La subred de sincronización puede reconfigurarse en caso de fallos o que no se alcance algún servidor.

Funciona sobre UDP.



Coordinación

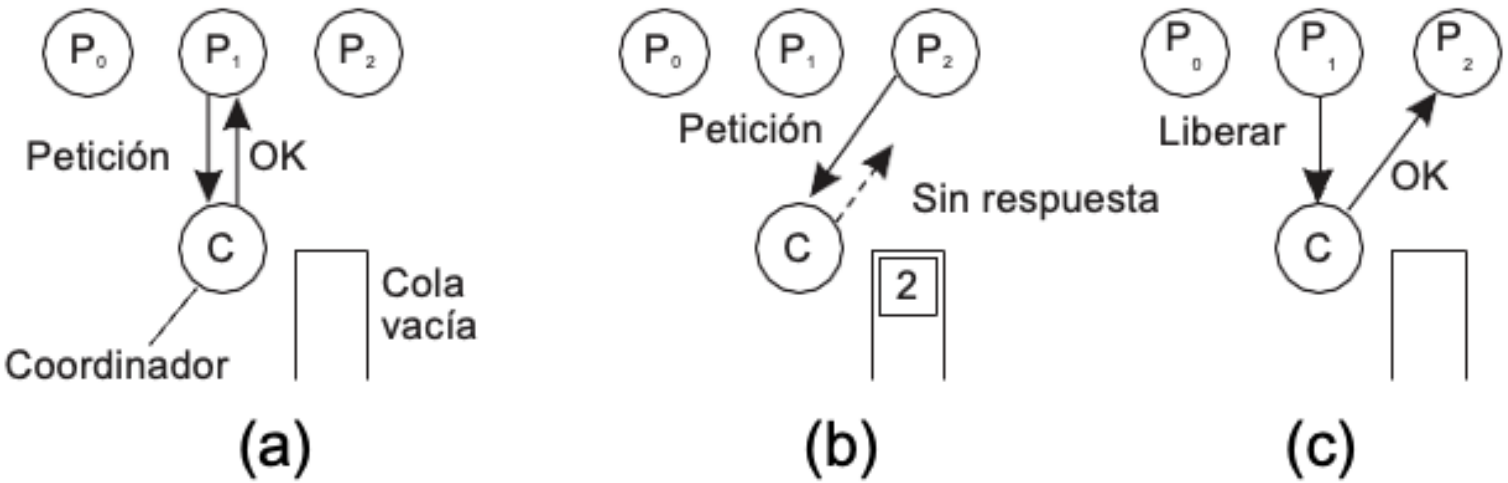
1. Sincronización de reloj
   - Algoritmo de Cristian
   - NTP
2. Exclusión mutua
   - Algoritmo centralizado
   - Algoritmo distribuido
   - Algoritmo *token ring*

Exclusión mutua

Vamos a ver algunos ejemplos de **algoritmos** para alcanzar la **exclusión mutua** entre una colección de **procesos**, de forma que **coordinan** el acceso a **recursos compartidos** en **sistemas distribuidos**.

- Basados en permisos (necesita permiso de otro proceso).
  - Algoritmo centralizado
  - Algoritmo distribuido
- Basados en *token* (un *token* es pasado entre los procesos).
  - Algoritmo *token ring*

Algoritmo centralizado



a) El proceso P_1 solicita al coordinador permiso para acceder al recurso compartido. El permiso es dado.

b) El proceso P_2 entonces solicita permiso para acceder al mismo recurso. El coordinador no responde.

c) Cuando P_1 libera el recurso, se lo dice al coordinador y éste responde a P_2.

Algoritmo centralizado

Ventajas:

- **Requiere pocos mensajes** para completar la gestión del acceso a una sección crítica.
- El **coordinador sabe** si la sección crítica está libre u ocupada, y en función de ello sólo permite que un único proceso la ejecute en cada momento.

Desventajas:

- La **comunicación debe ser fiable**, pues la pérdida de algún mensaje puede tener efectos graves. Por ejemplo, la pérdida de un mensaje de petición implicaría que el solicitante correspondiente permanezca suspendido indefinidamente.
- El **proceso coordinador constituye un punto único de fallo**. Con la caída del proceso coordinador se perdería toda la información relacionada con el estado actual de la sección crítica y la identidad de su proceso propietario.

Algoritmo distribuido

1- Cuando un **proceso** quiere **acceder** a la **sección crítica**, difundirá un **mensaje** INTENTAR a **todos** los **procesos** del sistema.

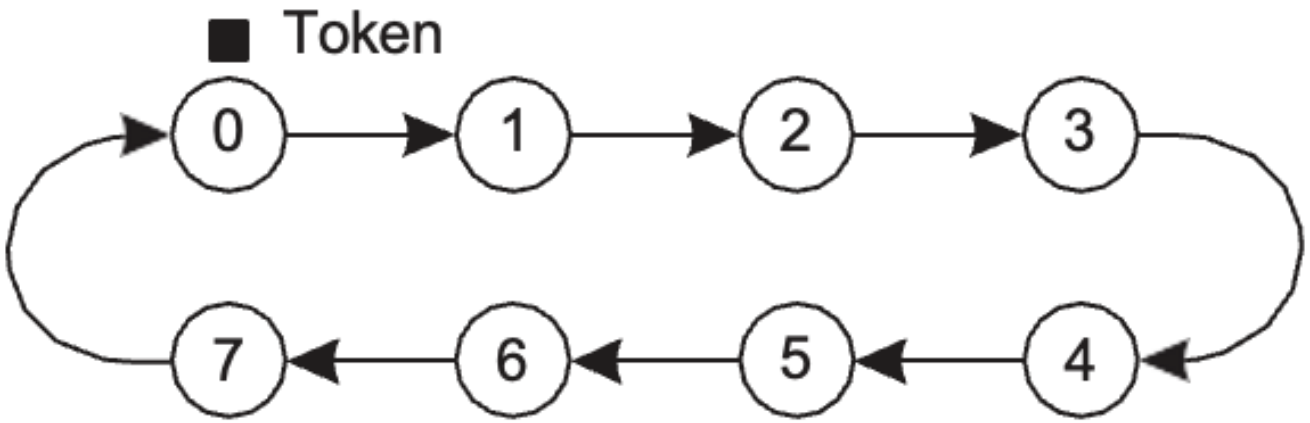
2- Cuando un **proceso reciba** un **mensaje** INTENTAR, actuará de la siguiente forma:

- Si no está en la sección crítica ni intentaba entrar, responderá con un mensaje OK.
- Si está en la sección crítica, no contesta, pero encola el mensaje.
- Si no está en la sección crítica, pero quiere entrar, compara el número de evento del mensaje entrante con el que él mismo envió al resto. Vence el número más bajo.
  - Si el mensaje entrante es más bajo, el receptor responde OK.
  - Si fuera más alto, no responde y encola el mensaje.

3- Un **proceso entrará** en la **sección crítica** cuando **recibe OK** de **todos** (todos aceptan la entrada). Cuando abandone la sección, enviará OK a todos los procesos que enviaron los mensajes que retuvo en su cola (avisa la salida).

Algoritmo *token ring*

- Organiza los procesos en un anillo lógico.
- Permite que un *token* se pase entre todos los procesos.
- El que tiene el *token* puede acceder a la región crítica (si quiere).



Contenido

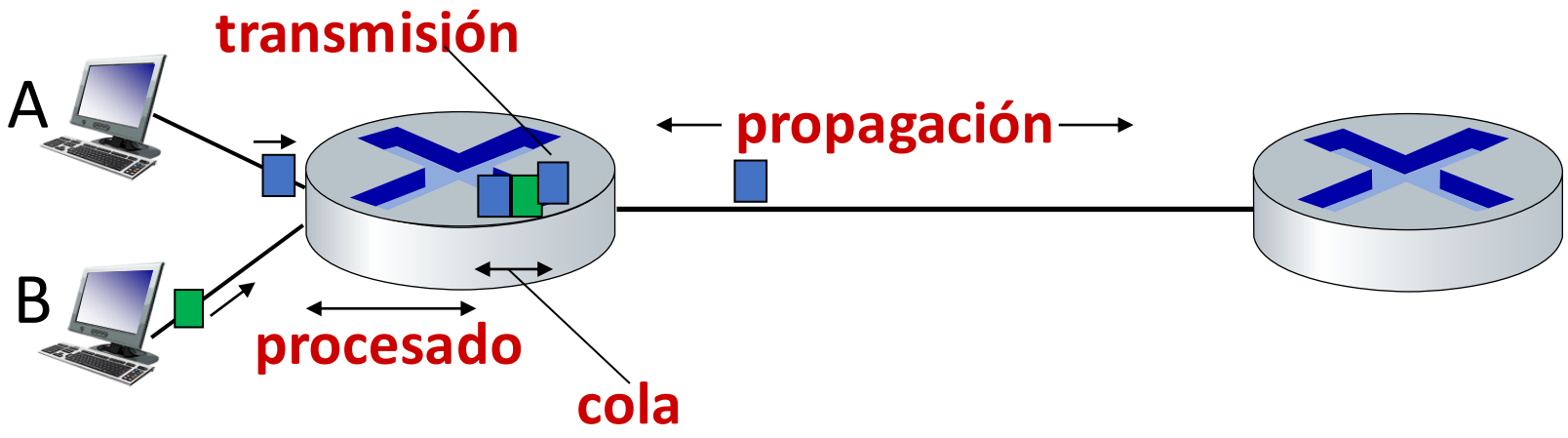
1. Caracterización de los sistemas distribuidos
2. Retos de los sistemas distribuidos
3. Arquitectura
4. Coordinación
5. Estimación de retrasos en la red

Retrasos en redes de conmutación de paquetes

Vamos a estudiar cuatro posibles tipos de **retardos de comunicación** en un sistema distribuido:

1. Retardo de **transmisión**
2. Retardo de **propagación**
3. Retardo de **procesado**
4. Retardo en **cola**

Retrasos en redes de conmutación de paquetes



$$d_{nodo} = d_{proc} + d_{cola} + d_{trans} + d_{prop}$$

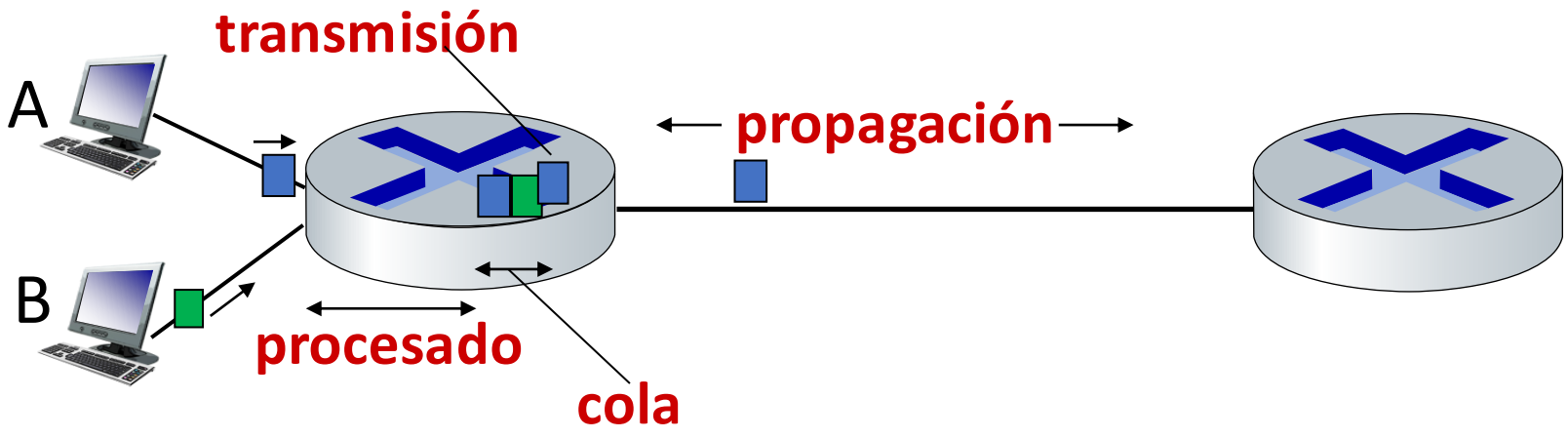
d_{proc}: retraso de procesado

- Comprobar error bits
- Determinar enlace de salida
- Del orden de microsegundos

d_{cola}: retraso de cola

- Tiempo esperando en el enlace de salida para transmisión
- Depende del nivel de congestión del *router*

Retrasos en redes de conmutación de paquetes



$$d_{\text{nodo}} = d_{\text{proc}} + d_{\text{cola}} + d_{\text{trans}} + d_{\text{prop}}$$

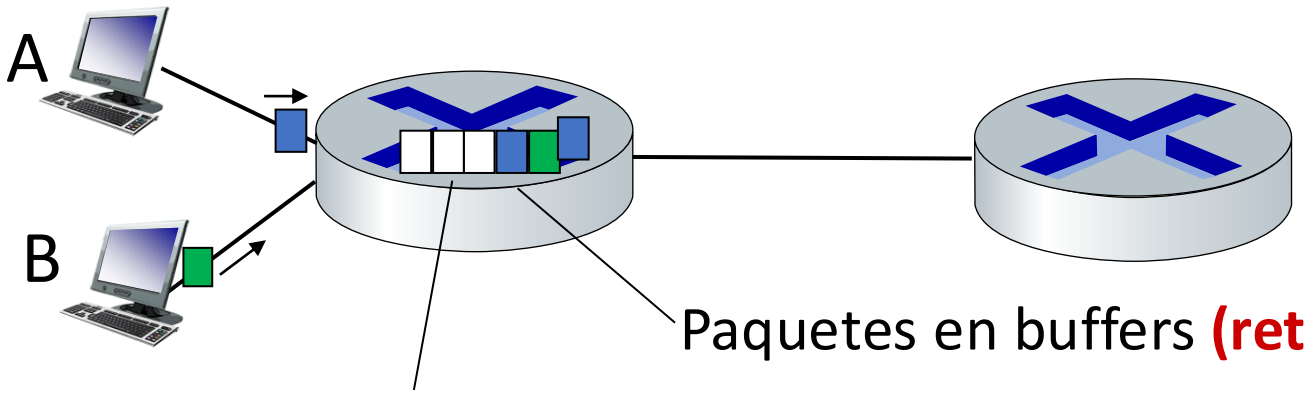
d_{trans}: retraso de transmisión

- L: longitud del paquete (bits)
- R: tasa transmisión enlace *(bps)*
- $d_{trans} = L/R$

d_{prop}: retraso de propagación

- d: longitud física del enlace
- v: velocidad propagación
- $d_{\text{prop}} = d/v$

Perdida de paquetes

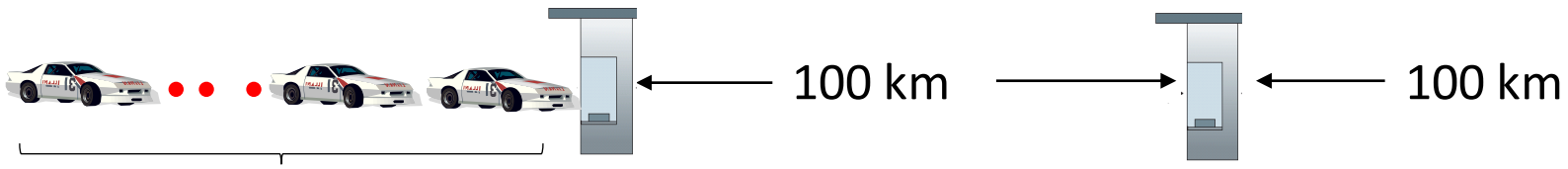


Paquetes en buffers **(retraso de cola)**

Cola de paquetes: cuando **no hay disponibilidad** los **paquetes** se **pierden**

Comparación retraso transmisión y propagación

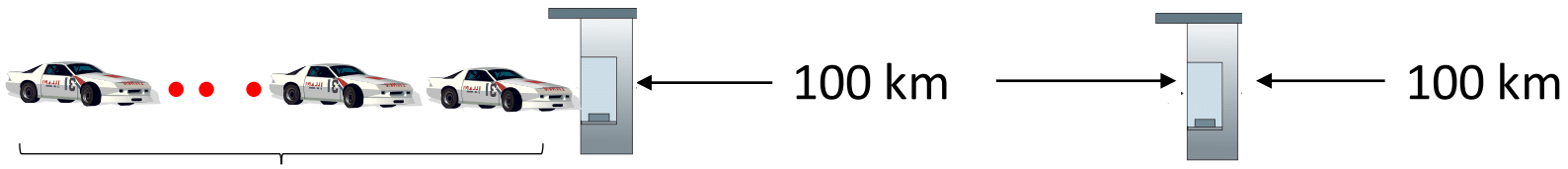
Analogía caravana de coches



10 coches = caravana = paquete
Coche = bit. Los coches se propagan a 100 km/h
El peaje tarda 12 s en servir a un coche (retardo de transmisión)
¿Cuánto tiempo tardará hasta que la caravana esté por completo ante el segundo peaje?

Comparación retraso transmisión y propagación

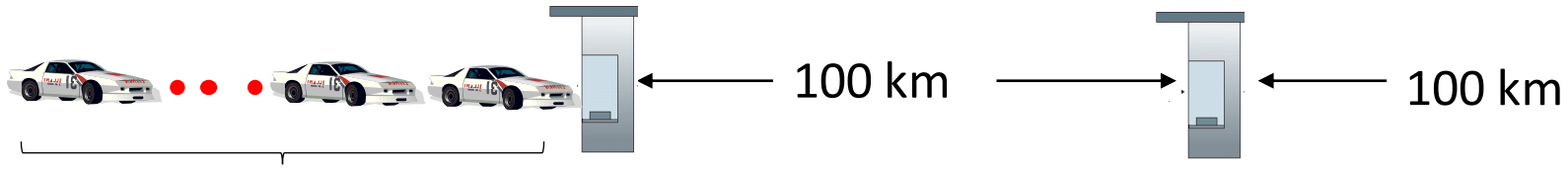
Analogía caravana de coches



10 coches = caravana = paquete
Coche = bit. Los coches se propagan a 100 km/h
El peaje tarda 12 s en servir a un coche (retardo de transmisión)
¿Cuánto tiempo tardará hasta que la caravana esté por completo ante el segundo peaje?
12x10=120 s=2 min
100km/(100km/h)=1 h=60 min
Respuesta: 62 min

Comparación retraso transmisión y propagación

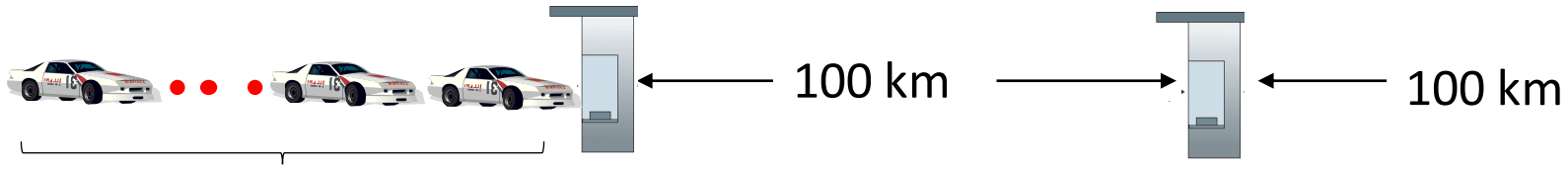
Analogía caravana de coches



10 coches = caravana = paquete
Coche = bit. Los coches se propagan a 1000 km/h
El peaje tarda 1 min en servir a un coche (retardo de transmisión)
¿Llegarán coches al segundo peaje antes de que se de servicio a todos los coches en el primer peaje?

Comparación retraso transmisión y propagación

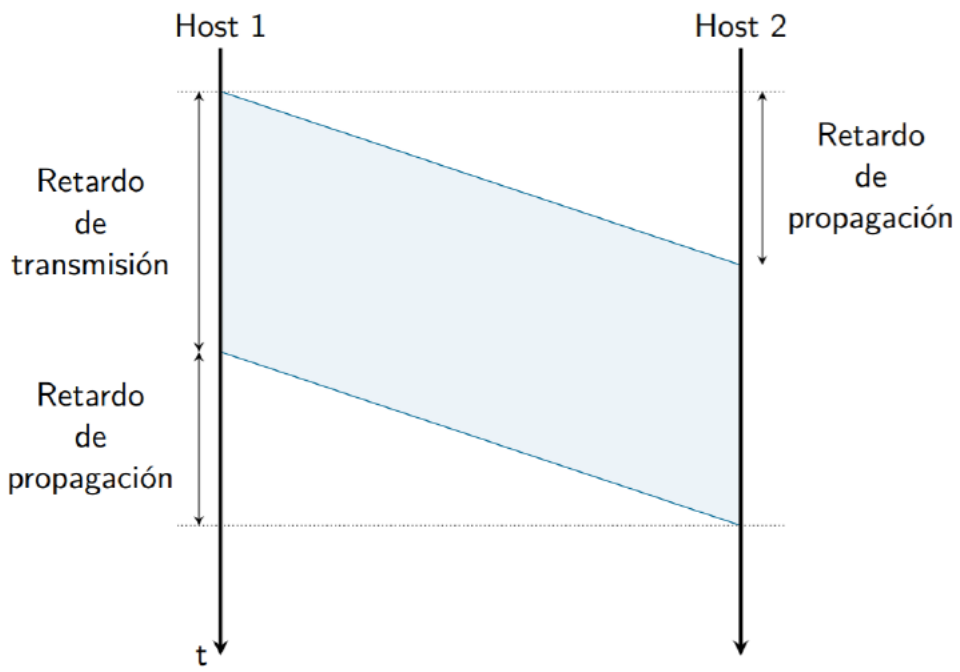
Analogía caravana de coches



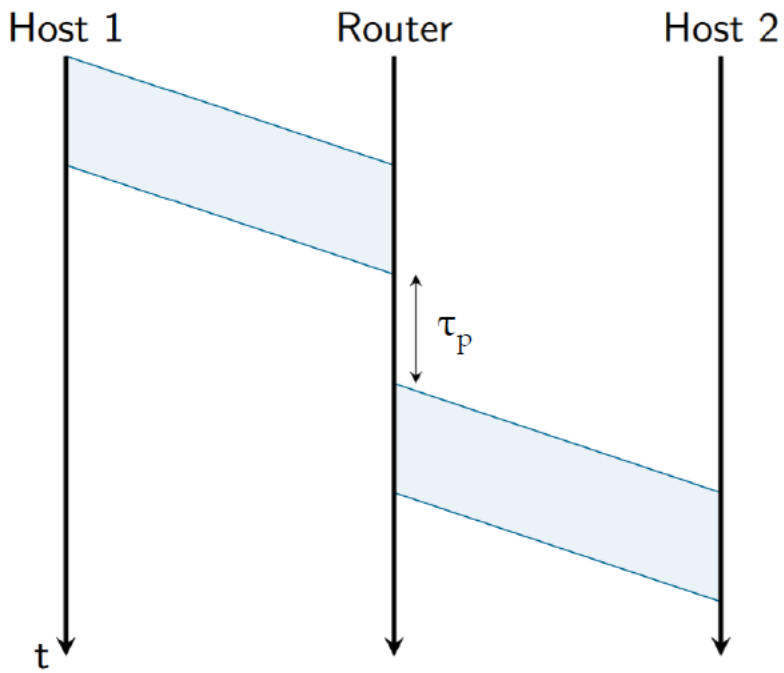
10 coches = caravana = paquete
Coche = bit. Los coches se propagan a 1000 km/h
El peaje tarda 1 min en servir a un coche (retardo de transmisión)
¿Llegarán coches al segundo peaje antes de que se de servicio a todos los coches en el primer peaje?

Sí, después de 7 min el primer coche llega mientras tres coches están aún en el primer peaje

Retrasos: representación



Retrasos de transmisión y propagación



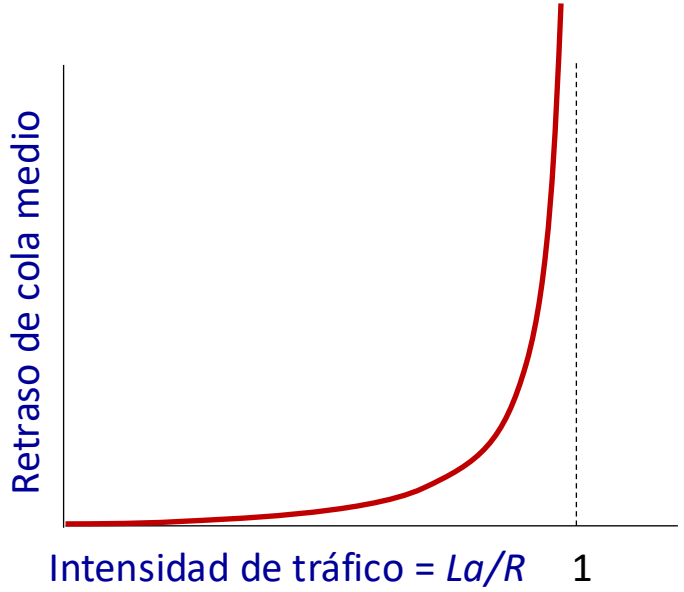
Retraso de procesado

Retraso de cola

- *a:* tasa de llegada de paquetes media
- *L:* longitud del paquete (bits)
- *R:* ancho de banda del enlace (tasa de transmisión)

$$\frac{L \cdot a}{R} : \frac{\text{Tasa llegada de bits}}{\text{Tasa de servicio de bits}}$$

"Intensidad de tráfico"



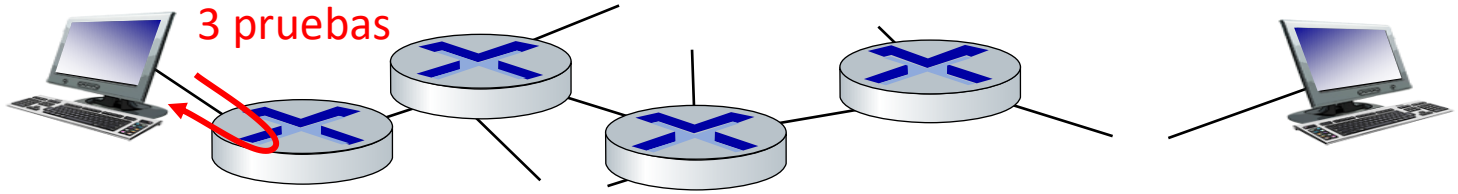
- ***La/R ~ 0:*** retraso medio en cola pequeño
- ***La/R -> 1:*** retraso medio en cola grande
- ***La/R > 1:*** más "trabajo" llegando es más de lo que puede ser servido - ¡retraso medio infinito!

La/R ~ 0

La/R -> 1

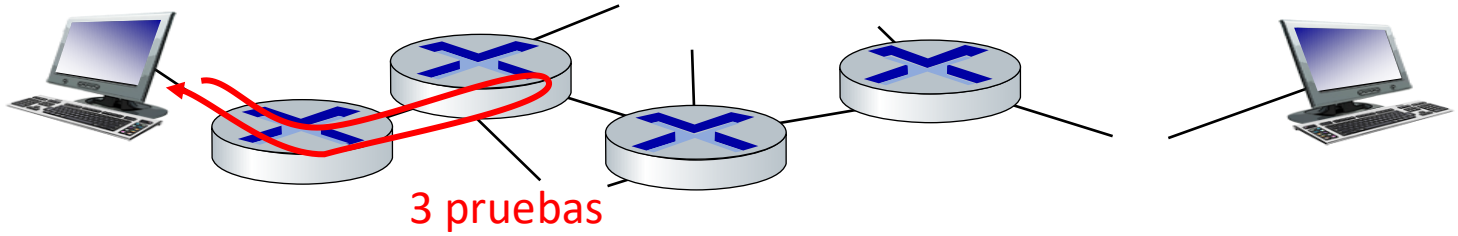
Retrasos “reales”

- **Traceroute** (RFC 1393): proporciona medidas de retraso desde la fuente a los *routers* en el camino hacia el destino.
- Envía tres paquetes que alcanzarán al *router* i de la ruta hacia el destino (con el campo TTL con valor i).
- El *router* i devuelve paquetes al origen.
- El *host* origen mide el intervalo de tiempo entre la transmisión y la respuesta.



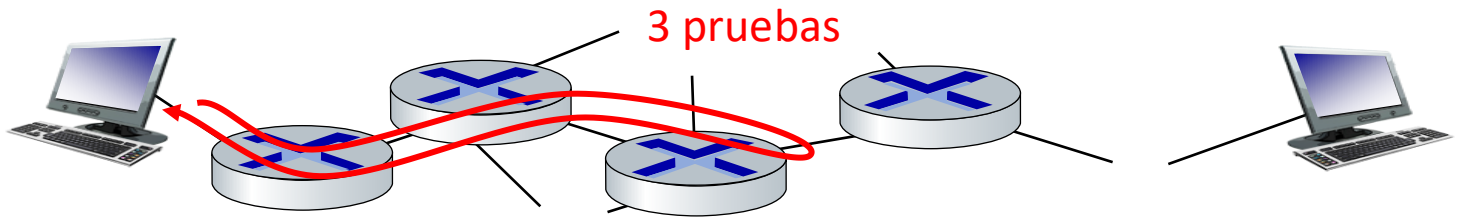
Retrasos “reales”

- **Traceroute** (RFC 1393): proporciona medidas de retraso desde la fuente a los *routers* en el camino hacia el destino.
- Envía tres paquetes que alcanzarán al *router* i de la ruta hacia el destino (con el campo TTL con valor i).
- El *router* i devuelve paquetes al origen.
- El *host* origen mide el intervalo de tiempo entre la transmisión y la respuesta.



Retrasos “reales”

- **Traceroute** (RFC 1393): proporciona medidas de retraso desde la fuente a los *routers* en el camino hacia el destino.
- Envía tres paquetes que alcanzarán al *router* i de la ruta hacia el destino (con el campo TTL con valor i).
- El *router* i devuelve paquetes al origen.
- El *host* origen mide el intervalo de tiempo entre la transmisión y la respuesta.



Retrasos “reales”

traceroute: gaia.cs.umass.edu a www.eurecom.fr

3 medidas de retraso de gaia.cs.umass.edu a cs-gw.cs.umass.edu

3 medidas de retraso a border1-rt-fa5-1-0.gw.umass.edu

```
1  cs-gw (128.119.240.254)  1 ms  1 ms  2 ms
2  border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145)  1 ms  1 ms  2 ms
3  cht-vbns.gw.umass.edu (128.119.3.130)  6 ms 5 ms 5 ms
4  jn1-at1-0-0-19.wor.vbns.net (204.147.132.129)  16 ms 11 ms 13 ms
5  jn1-so7-0-0-0.wae.vbns.net (204.147.136.136)  21 ms 18 ms 18 ms
6  abilene-vbns.abilene.ucaid.edu (198.32.11.9)  22 ms  18 ms  22 ms
7  nycm-wash.abilene.ucaid.edu (198.32.8.46)  22 ms  22 ms  22 ms
8  62.40.103.253 (62.40.103.253)  104 ms 109 ms 106 ms
9  de2-1.de1.de.geant.net (62.40.96.129)  109 ms 102 ms 104 ms
10  de.fr1.fr.geant.net (62.40.96.50)  113 ms 121 ms 114 ms
11  renater-gw.fr1.fr.geant.net (62.40.103.54)  112 ms  114 ms  112 ms
12  nio-n2.cssi.renater.fr (193.51.206.13)  111 ms  114 ms  116 ms
13  nice.cssi.renater.fr (195.220.98.102)  123 ms  125 ms  124 ms
14  r3t2-nice.cssi.renater.fr (195.220.98.110)  126 ms  126 ms  124 ms
15  eurecom-valbonne.r3t2.ft.net (193.48.50.54)  135 ms  128 ms  133 ms
16  194.214.211.25 (194.214.211.25)  126 ms  128 ms  126 ms
17  * * *
18  * * *
19  fantasia.eurecom.fr (193.55.113.142)  132 ms  128 ms  136 ms
```

Universidad
LOYOLA

Universidad
LOYOLA